

Online Real Estate System - Interview Questions & Answers

This document provides a comprehensive list of potential interview questions based on the architecture, features, and technical challenges of your **Online Real Estate System**, along with detailed, professional answers to help you prepare.

1. Project Differentiation

Question: There are many established real estate platforms (like Zillow, MagicBricks, etc.). What makes your project stand out, and why did you build it?

Answer: While large platforms exist, my goal was to build a comprehensive, end-to-end ecosystem rather than just a basic property listing site. What differentiates my project is the **technical depth and complex business logic** I implemented:

- **Integrated Financial Services:** Unlike basic platforms, I built a complete Loan Application engine. This includes a state-machine for loan approvals, automated EMI schedule generation, and dynamic PDF generation for loan agreements.
- **Real-Time Communication:** I implemented a real-time chat system using WebSockets (STOMP/SockJS) allowing buyers and sellers to negotiate directly without relying on third-party messaging apps.
- **Robust Fallback Mechanisms:** For property mapping, I integrated Leaflet and OpenStreetMap. Instead of a naive geocoding implementation that fails on slight misspellings, I engineered a progressive fallback mechanism (trying street, then city, then state) with rate-limiting protection.
- **Authentication:** I built a dual-authentication system supporting both traditional JWT-based login and Google OAuth 2.0.

This project was built to demonstrate my ability to handle complex state management, integrate third-party APIs, and build production-ready, full-stack architectures.

2. Architecture & Tech Stack

Question: Can you walk me through the architecture and technology stack of your application?

Answer: The application follows a modern decoupled client-server architecture:

- **Frontend:** Built with React.js. It manages complex UI state using Hooks and communicates with the backend via Axios REST calls and STOMP over WebSockets.
 - **Backend:** Built with Java and Spring Boot. I used Spring Web for REST APIs, Spring Data JPA (Hibernate) for ORM, and Spring Security for protecting endpoints.
 - **Database:** PostgreSQL is used as the primary relational database to enforce strong data integrity, especially critical for the financial (Loan) aspects of the application.
 - **Security:** Stateless JWT (JSON Web Tokens) for authentication.
 - **File Storage:** Local multipart file upload for property images and loan documents, served securely via Spring's resource handlers.
-

3. Real-Time Chat (WebSockets)

Question: How did you implement the real-time chat feature, and what challenges did you face?

Answer: I used **WebSockets** with the **STOMP** protocol, backed by **SockJS** for fallback support in environments where raw WebSockets are blocked.

- **Implementation:** In Spring Boot, I configured a message broker to handle routing messages to specific users. On the React side, I used `@stomp/stompjs` to subscribe to user-specific channels.
- **Challenge & Resolution:** One major challenge was handling connection drops and reconnections. Initially, the frontend would fail to reconnect because the `SockJS` instance was being reused after a connection closed. I solved this by restructuring the WebSocket factory to instantiate a brand new `SockJS` client on every reconnection attempt, and by binding the frontend UI state directly to the underlying `client.connected` flag to ensure the UI always reflected the true network state.

4. Authentication & OAuth 2.0

Question: How is authentication handled in your system, particularly the Google Login integration?

Answer: The core authentication is stateless, using **JWTs (JSON Web Tokens)**. When a user logs in, the Spring Boot backend generates a JWT which the React frontend stores and attaches to the `Authorization` header of subsequent requests. For **Google OAuth 2.0**, I used the Google Identity Services (GSI) client on the frontend.

- When a user clicks "Sign in with Google," Google returns an ID token to the frontend.
- Instead of blindly trusting the frontend, I send this ID token to my Spring Boot backend.
- The backend securely verifies the token's signature using Google's `tokeninfo` endpoint. If valid, the backend automatically creates a local user account (if it's their first time) and issues my application's internal JWT, keeping the session fully integrated with my Spring Security configuration.

5. Complex Business Logic (Loan Processing)

Question: The Loan Application feature sounds complex. Can you explain the workflow and how you managed the state?

Answer: The loan feature functions as a state machine. The lifecycle is:

1. **PENDING:** User submits basic details.
2. **APPROVED:** Admin approves the initial request.
3. **PROCESSING:** User submits Bank Details.
4. **DOCS_VERIFIED:** User uploads required KYC documents (Aadhaar, PAN, Salary Slip). The UI dynamically filters out already-uploaded documents to prevent duplicates.
5. **DISBURSED:** Admin verifies the documents and disburses the funds.
6. **COMPLETED:** User pays off all auto-calculated EMI payments.

I handled the state via JPA entities and REST endpoints. A significant technical challenge here was the **JSON Infinite Recursion** problem. Because a `LoanApplication` has a `@OneToMany` relationship with `EmiPayment`, and `EmiPayment` has a `@ManyToOne` back to the loan, fetching the loan caused an infinite serialization loop. I solved this by strategically applying `@JsonIgnore` to the child references to break the loop while maintaining the database relationships.

6. Geocoding & Mapping

Question: How did you implement property locations on the map, and how did you handle geocoding failures?

Answer: I used **React-Leaflet** rendering OpenStreetMap tiles. To convert text addresses into map coordinates (Lat/Lng), I used the Nominatim geocoding API. Because geocoding can be unreliable (users type addresses differently) and APIs have strict rate limits, I built a **Progressive Fallback Mechanism**:

- The system first attempts to geocode the exact street address.
- If that fails or returns zero results, the system catches the error and automatically falls back to searching just the City and State.
- If that also fails, it centers the map on a default geographic region. I also implemented artificial delays (debouncing/throttling) between geocoding requests to ensure my application wasn't blocked by the API provider for exceeding rate limits.

7. Database Integrity & Troubleshooting

Question: Can you describe a database-related issue you encountered and how you fixed it?

Answer: During the development of the Loan Application module, I encountered a `SQLState: 22001 (value too long for type character varying)` exception. I had introduced a new loan status called `"DOCUMENT_VERIFICATION"`, which is 21 characters long. However, the database column for status was configured with `VARCHAR(20)`. While I could have run an `ALTER TABLE` SQL command to increase the column size, I realized this could cause deployment friction in different environments. Instead, I refactored the backend and frontend code to use a shorter, identical alias: `"DOCS_VERIFIED"`. To ensure backwards compatibility with any existing records, I updated the React frontend to normalize both the old and new strings, ensuring the UI (like the progress bar) wouldn't break for older records.